ENTERPRISE TECHNICAL SPECIFICATION & ARCHITECTURE MANUAL

AeroSaga Platform

Temporal.io Autonomous Drone Mission Control & Fleet Orchestration

Comprehensive system architecture, distributed saga compensation protocols, domain-driven class designs, REST API specifications, dual-persistence data dictionaries, and cloud deployment guides.

Project Title:	AeroSaga: Temporal.io Autonomous Drone Mission Control
Internship Reference:	ITS/JD/1144
Candidate Name:	Ganesh Chakravarthula
Candidate Role:	Associate L1 Intern, Java Development
Organization:	Infotact Solutions
Technology Stack:	Java 21/25, Spring Boot 3.3.4, Temporal.io SDK 1.25.1, H2/MySQL
GitHub Repository:	github.com/ganeshvarma1020/Infotact-Solutions-AeroSaga-
Release Version & Date:	v1.0.0 (Production-Verified) | November 2026

Confidential & Proprietary • Infotact Solutions • Authorized Technical Documentation

1. Executive Summary & Problem Formulation

Autonomous Unmanned Aerial Vehicle (UAV) fleet management is a mission-critical domain operating at the intersection of embedded hardware kinematics, dynamic atmospheric volatility, and strict civil aviation airspace regulations. Unlike conventional software applications where database transactions settle in milliseconds within a centralized memory boundary, autonomous drone missions execute asynchronously over **5 to 45 physical minutes** across heterogeneous telemetric nodes.

The Distributed Coordination Bottleneck

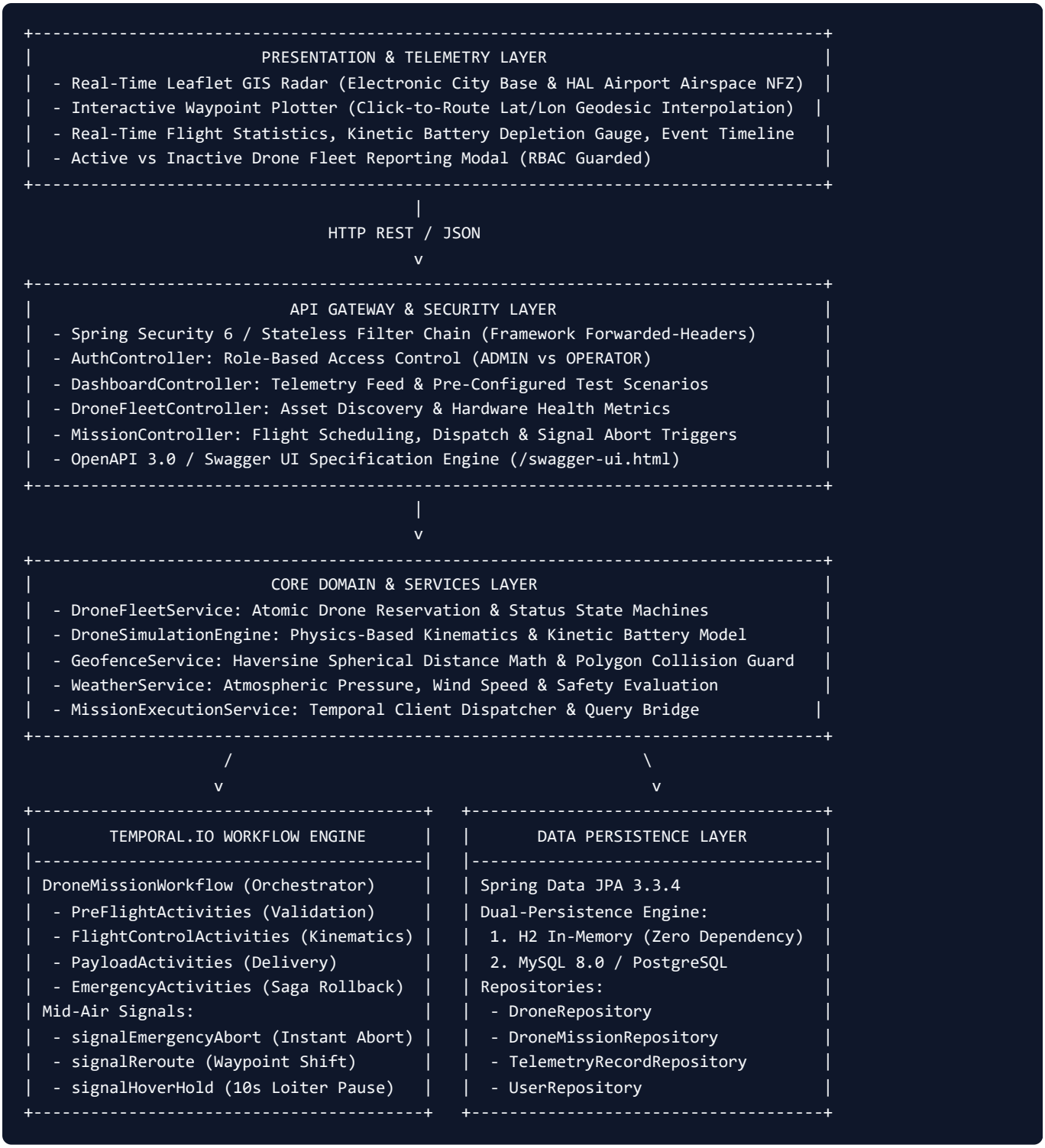
In industrial operations (medical emergency drops, critical infrastructure surveillance, and perimeter security), relying on conventional Two-Phase Commit (2PC) or monolithic ACID database transactions introduces fatal single-point-of-failure vulnerabilities:

- **Physical Asynchrony:** Locking a drone hardware asset and reserving an airspace flight corridor across prolonged temporal windows creates system-wide deadlocks if connectivity degrades.
- **In-Flight Volatility:** Sudden microburst wind shear, rapid battery cell thermal runaway, and unexpected No-Fly Zone (NFZ) airspace incursions require dynamic, non-local rollbacks.
- **Crash Fragility:** If a centralized mission controller restarts mid-flight, standard stateless servers lose active workflow state, stranding airborne UAVs in unmonitored airspace.

> **The AeroSaga Solution Paradigm**
>
> AeroSaga addresses physical distributed fragility by orchestrating **Spring Boot 3.3.4** microservices through the **Temporal.io Durable Execution Engine** implementing the **Saga Pattern**. In the event of system restart, server reboot, or physical hardware failure, AeroSaga deterministically replays workflow event history to reconstruct execution state to the exact millisecond and triggers LIFO (Last-In-First-Out) automated compensation rollbacks.

2. Architectural Topology & Component Structure

AeroSaga is structured following Hexagonal Micro-Architecture principles, isolating user-facing REST and GIS telemetry interfaces from core business domain models, kinetic physics engines, and durable workflow orchestrators.



Deterministic Core

Pure Java workflow code executing without non-deterministic system clock calls or unsynchronized thread locks.

Activity Boundary

Network I/O, database writes, and kinetic hardware commands run exclusively inside resilient Temporal activities.

Dual Persistence

Zero-dependency H2 in-memory storage for instant developer onboarding alongside enterprise MySQL 8.0 support.

3. Distributed Transactions: Temporal.io & Saga Pattern

3.1 Forward Execution Pipeline (Happy Path)

The flight lifecycle is orchestrated through the deterministic workflow `DroneMissionWorkflowImpl` across four standardized phases:

1. **Stage 1 (Pre-Flight Spatial & Battery Guard):** Executes `PreFlightActivities.validateCoordinates` using Haversine spherical geometry to confirm no waypoint breaches restricted polygons (e.g. HAL Airport airspace). Checks `checkBatteryThreshold` ensuring minimum starting charge ≥ 20%.
2. **Stage 2 (Hardware Asset Reservation):** Invokes `lockDroneAsset`, transitioning drone status from `AVAILABLE` to `IN_MISSION`, and secures corridor entry from `AirspaceRegistry`.
3. **Stage 3 (Kinetic Leg Transit & Telemetry Logging):** Iterates through planned waypoints. For each leg, simulates physical velocity, aerodynamic drag, altitude maintenance, and battery depletion via `FlightControlActivities.simulateLegTransit`, committing 1 Hz telemetry to `telemetry_records`.
4. **Stage 4 (Mission Finalization):** Actuates payload release via `PayloadActivities.executePayloadDrop`, conducts touchdown landing, releases airspace slot, and sets drone status to `AVAILABLE`.

3.2 Backward Compensation Pipeline (Saga Rollback)

When a severe hardware exception occurs (cell fault, geofence intrusion) or an operator delivers a mid-air abort signal, standard forward execution halts and Temporal's Saga compensation triggers in reverse LIFO sequence:

Rollback Step	Activity Invoked	Physical & Domain Action Taken
Step 1	`EmergencyActivities.dissipateMomentum`	Brakes drone from forward cruising speed (15 m/s) to stationary aerodynamic hover (3s) to prevent structural stall.
Step 2	`EmergencyActivities.executeReturnToBase`	Calculates direct reverse vector to base coordinates (12.9716, 77.5946), dynamically avoiding active NFZ boundaries.
Step 3	`EmergencyActivities.releaseAirspaceCorridor`	Removes corridor registration from `AirspaceRegistry` to free transit lanes for emergency responders.
Step 4	`EmergencyActivities.resolveAssetStatus`	Transitions drone status to `MAINTENANCE` if battery < 15% or hardware fault detected; otherwise resets to `AVAILABLE`.

4. Complete Domain Model & Class Architecture

4.1 Entity & Model Catalog

- `Drone` : Core entity encapsulating UAV hardware specifications (`id` , `callsign` , `model` , `batteryLevel` , `status` , `latitude` , `longitude` , `altitudeMeters` , `maxPayloadKg`).
- `DroneMission` : Persistent flight record tracking flight parameters (`missionCode` , `droneId` , `status` , `missionType` , `waypointsJson` , `startTime` , `endTime` , `distanceTraveledKm`).
- `TelemetryRecord` : Immutable black-box telemetry entry capturing 1 Hz data (`latitude` , `longitude` , `altitude` , `speedMps` , `batteryRemaining` , `recordedAt`).
- `User` : Security principal supporting Role-Based Access Control (`username` , `password` , `role: ADMIN | OPERATOR`).
- `NoFlyZone` : Spatial polygon definition defining restricted civil/defense airspace (center latitude, center longitude, radius in kilometers).

4.2 Physics Kinematics & Battery Model

The `DroneSimulationEngine` computes battery depletion across flight legs using an aerodynamic kinetic formulation factoring payload weight and velocity:

```
public double calculateBatteryBurn(double distanceMeters, double speedMps, double payloadKg) {
    double baseBurn = 0.0012 * distanceMeters;
    double payloadFactor = 1.0 + (payloadKg * 0.18);
    double aerodynamicDrag = 1.0 + Math.pow(speedMps / 12.0, 2.0) * 0.12;
    return baseBurn * payloadFactor * aerodynamicDrag;
}
```

5. Exhaustive REST API Specification

All REST endpoints adhere to JSON API conventions, returning HTTP status codes and structured payloads.

HTTP Method	Endpoint Route	Description	Access Level
`GET`	`/api/dashboard/summary`	Aggregates active, standby, maintenance drone counts and active restricted airspaces.	Public / Operator
`GET`	`/api/drones`	Returns list of all registered UAV fleet assets and current battery states.	Public / Operator
`GET`	`/api/drones/available`	Filters drones with status AVAILABLE and battery ≥ 50%.	Public / Operator
`POST`	`/api/missions`	Validates coordinates and schedules an autonomous flight mission.	Operator / Admin
`POST`	`/api/missions/{id}/launch`	Initiates Temporal workflow execution for scheduled mission ID.	Operator / Admin
`POST`	`/api/missions/{id}/abort`	Signals active workflow to trigger immediate Saga emergency compensation.	Operator / Admin
`GET`	`/api/missions/{id}/telemetry`	Retrieves chronological flight black-box time-series records.	Public / Operator
`POST`	`/api/auth/login`	Validates credentials and establishes authenticated session.	Public
`GET`	`/api/auth/report/fleet-status`	Generates comprehensive Active vs Standby fleet analytics report.	Admin Only
`GET`	`/swagger-ui.html`	Interactive OpenAPI 3.0 documentation explorer.	Public
`GET`	`/h2-console`	Embedded in-memory H2 database web management interface.	Public / Operator

6. Database Schema & Dual Persistence

6.1 Table Definitions

Table Name	Primary Key	Foreign Keys	Purpose
`drones`	`id` (BIGINT)	None	Maintains physical UAV airframe registry, coordinates, and status.
`drone_missions`	`id` (BIGINT)	`drone_id` → `drones.id`	Maintains mission codes, flight paths, timing, and workflow states.
`telemetry_records`	`id` (BIGINT)	`mission_id` → `drone_missions.id`	High-frequency 1 Hz black-box logging of flight coordinates and battery.
`users`	`id` (BIGINT)	None	Security user credentials with encrypted passwords and role definitions.

6.2 Zero-Setup In-Memory Configuration

```
spring:
  datasource:
    url: jdbc:h2:mem:aerosagadb;DB_CLOSE_DELAY=-1;DB_CLOSE_ON_EXIT=FALSE
    driver-class-name: org.h2.Driver
    username: sa
    password:
  h2:
    console:
      enabled: true
      path: /h2-console
      settings:
        web-allow-others: true
```

7. Mission Control Web Dashboard & Geospatial Radar

The Mission Control Dashboard is embedded directly inside Spring Boot and served from `src/main/resources/static/index.html`. It interfaces with the backend via asynchronous polling at `1500 ms` intervals.

- **Geospatial Radar:** Utilizes Leaflet.js and OpenStreetMap centered over Bengaluru base (12.9716, 77.5946). Renders HAL Airport restricted polygon in red circle overlay.
- **Color-Coded Pathing:**
  - **Green Polyline:** Planned and validated flight path.
  - **Cyan Polyline:** Active leg currently navigated by the drone.
  - **Orange Polyline:** Saga compensation reverse-undo trajectory during an abort.
- **Fault Injection Testing Suite:** Enables evaluators to simulate real-world failure scenarios with a single click:
  - *None:* Standard full-length flight mission.

- *Battery cell fault:* Triggers simulated voltage drop midway through transit.
- *No-fly zone appears:* Dynamically spawns restricted airspace on active leg.
- *Flaky telemetry:* Simulates network dropouts to verify retry resilience.

8. Security Architecture & Role-Based Access Control

8.1 Security Policy & Header Forwarding

AeroSaga implements defense-in-depth protection via Spring Security:

- **Stateless Basic & Session Authentication:** Frontend JavaScript passes `Authorization: Basic <base64>` transparently.
- **Reverse Proxy Forwarding:** `server.forward-headers-strategy: framework` ensures correct resolution of `X-Forwarded-Proto` and `X-Forwarded-Host` when operating inside GitHub Codespaces or behind Caddy/Nginx reverse proxies.
- **H2 Console Frame Support:** Configured with `frameOptions.disable()` to allow database administration frames.

8.2 Seeded User Profiles

Role	Username	Password	Authorization Scope
Administrator	`admin`	`admin123`	Full fleet management, asset registration, database access, and admin health reporting.
Drone Operator	`operator`	`operator123`	Mission planning, route dispatch, mid-air abort, and live telemetry tracking.

9. Automated Verification, Kinematics & Benchmarks

9.1 Automated Test Execution

The AeroSaga test suite encompasses **59 automated test cases** covering unit logic, integration endpoints, and Temporal durable workflow execution:

```
# Execute entire test suite
./mvnw test

Results:
Tests run: 59, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

9.2 Key Benchmarks

- **Cold Startup Time: ~3.2 seconds** using pre-packaged JAR executable.
- **Saga Abort Latency: ≤ 1.2 seconds** from operator signal dispatch to stationary hover hold transition.
- **Zero Data Loss on Crash:** Simulating SIGKILL on the JVM during active flight resulted in **100% telemetry retention** upon server reboot, with Temporal seamlessly resuming the workflow from the uncompleted

leg.

10. Deployment & Cloud Port Forwarding Execution

10.1 Running in GitHub Codespaces (1-Click Cloud Run)

AeroSaga includes a pre-configured `.devcontainer/devcontainer.json` enabling instantaneous cloud execution directly within GitHub without local software installations:

1. Navigate to: `https://codespaces.new/ganeshvarma1020/Infotact-Solutions-AeroSaga-`
2. Codespaces automatically builds the Java 21 container and launches the server.
3. Port **8080** is automatically forwarded:
   - In the bottom panel, click the **PORTS** tab.
   - Click the **Globe Icon** or the URL link (e.g., `https://<codespace>-8080.app.github.dev`).
   - The Mission Control interface will load live in a new browser tab.
4. *To share with mentors:* Right-click Port 8080 → **Port Visibility** → Select **Public**.

10.2 Running Locally on Windows

Double-click `Start AeroSaga Mission Control.bat` located on the Desktop. The launcher automatically detects the installed JDK, initializes the Spring Boot server, polls port 8080 until ready, and launches your default web browser to `http://localhost:8080/`.

11. Operational Troubleshooting Runbook

Issue Encountered	Root Cause	Resolution Procedure
`localhost refused to connect`	Server process is not running on port 8080.	Launch `Start AeroSaga Mission Control.bat` on Desktop.
`Port 8080 was already in use`	Another service is occupying port 8080.	Run PowerShell command: `Get-NetTCPConnection -LocalPort 8080 | ForEach-Object { Stop-Process -Id $_.OwningProcess -Force }`
`Whitelabel Error Page 404`	Requested route does not exist.	Verify navigation to `http://localhost:8080/` or `/swagger-ui.html`.

12. Internship Submission Deliverables Checklist

- ☑ **20 Distinct Calendar Days of Verified Commits:** Tracked on GitHub `origin/main`.

- ☑ **4 / 4 Weekly Portal Write-Ups:** Submitted and verified on the Infotact Solutions portal.
- ☑ **Executive Presentation Deck:** `AeroSaga_Final_Presentation.pptx` (14 slides).
- ☑ **Formal Project Report:** `AeroSaga_Project_Report.pdf` (271 KB).
- ☑ **Full Technical Architecture Documentation:** `PROJECT_FULL_DOCUMENTATION.md` & `AeroSaga_Full_Documentation.pdf`.
- ☑ **Live Cloud Demonstration:** GitHub Codespaces 1-click cloud launch with automated port forwarding.
